

Reviewer Pack — Minimal Hodge Index Bound for Communication Complexity

Entry fdd0d404947e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 01:31:39 UTC

Minimal Hodge Index Bound for Communication Complexity

Entry ID: fdd0d404947e **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 01:31:39 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every n -communication complexity problem, the minimal Hodge index of its associated polynomial vector space is upper-bounded by a function of $\log(n)$, such that $h(P) \leq c \cdot \log^2(n)$, where P is the set of variables involved in the communication complexity problem and $h(P)$ denotes the minimal Hodge index of the associated projective variety defined over $\text{GF}(2)$.

Rationale (proposer's reasoning):

The Hodge theory provides a way to measure the complexity of algebraic varieties, which could be related to the structural complexity of communication protocols. If such an upper bound holds, it would suggest that certain communication complexity problems have polynomially bounded algebraic complexity.

Taxonomy category: AlgebraicGeometryCommunicationComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 85d8d60e1220d4cb

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For an n -communication complexity problem, if the minimal Hodge index of its associated polynomial vector space over $\text{GF}(2)$ exceeds $c \cdot \log^2(n)$, where c is a constant and $\log^2(n)$ is based on the number of variables n , then the conjecture is falsified. Otherwise, if for all n up to 40 and using 30 random seeds, the Hodge index does not exceed $c \cdot \log^2(n)$, the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge index" AND "communication complexity" AND matrix rank - "algebraic geometry" AND Hodge theory AND communication complexity - "projective variety" AND minimal Hodge index AND GF(2) polynomial vector space

Top relevant hits considered: - [http://arxiv.org/abs/1409.1534v1] Algorithms in Real Algebraic Geometry: A Survey - [http://arxiv.org/abs/1611.00827v2] Geometric complexity theory and matrix powering - [http://arxiv.org/abs/2108.06263v1] Algebraic Geometry and Representation theory in the study of matrix multiplication complexity and other problems in theo - [http://arxiv.org/abs/2509.25273v1] Hodge Conjecture via Singular Varieties - [http://arxiv.org/abs/1607.01412v2] Pink's theory of Hodge structures and the Hodge conjecture over function fields - [http://arxiv.org/abs/2010.10655v1] Chen-Jiang decompositions for projective varieties, without Hodge modules

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def hodge_index(P):
        n = len(P)
        A = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i, n):
                count = 0
                for k in range(n):
                    if (P[i] & P[j]) & (1 << k):
                        count += 1
                A[i][j] = count
                A[j][i] = count
        return max(max(row) for row in A)

    def communication_complexity(n):
        # Example: a simple linear function to generate n-communication complexity instances
        return random.randint(1, n)

    total_metric_value = 0
```

```

instances_tested = 0
n_max = 0

for n in [5, 10, 15, 20, 30, 40]:
    if n > n_max:
        n_max = n

    for _ in range(5):
        P = {random.getrandbits(n) for _ in range(n)}
        cc = communication_complexity(n)
        h_P = hodge_index(P)

        total_metric_value += h_P
        instances_tested += 1

mean_metric_value = Fraction(total_metric_value, instances_tested)
conjecture_holds = all(h_P <= 2 * math.log(n) ** 2 for n in [5, 10, 15, 20, 30, 40] for _ in range(5))

return {
    "metric_name": "Hodge Index",
    "metric_value": mean_metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(res["metric_value"] for res in results) / len(results)
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

```

Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5b3318b7.py", line 72, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5b3318b7.py", line 49, in run_trial
    h_P = hodge_index(P)
           ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5b3318b7.py", line 28, in hodge_index
    if (P[i] & P[j]) & (1 << k):
        ~^^^
TypeError: 'set' object is not subscriptable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents a definitive evaluation of the conjecture. | next: Investigate and fix the error in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14014
2	preregistration	ollama_remote	glm4:latest	0	0	10088
3	novelty	ollama_remote	glm4:latest	0	0	11717
4	novelty	ollama_remote	glm4:latest	0	0	9517
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	73249
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9502
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14177
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8676
9	judge	ollama_remote	glm4:latest	0	0	21480

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 172419 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/fdd0d404947e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/fdd0d404947e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/fdd0d404947e.tar.gz` (if generated)